



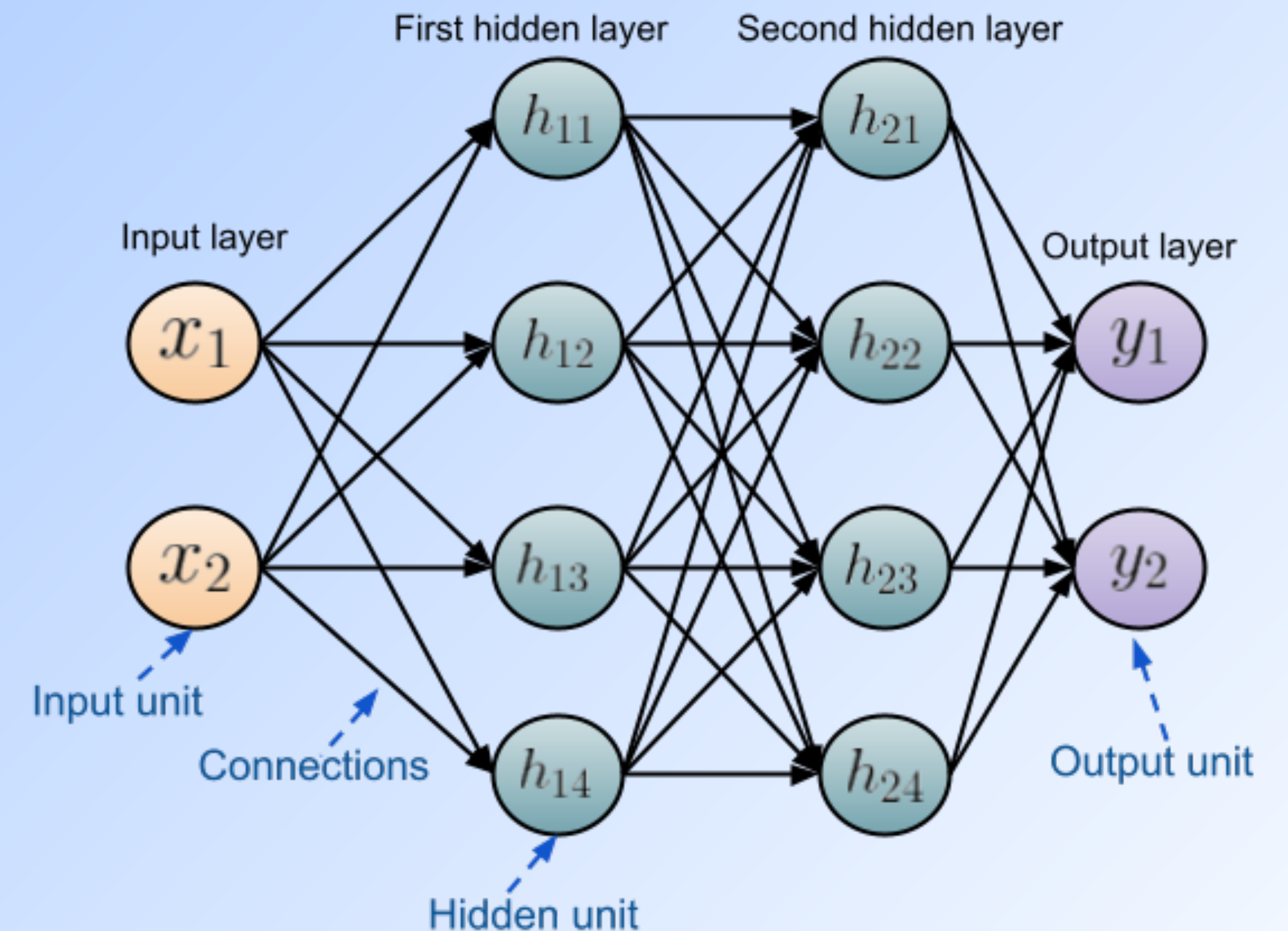
AI61002: Deep Learning Foundations and Applications

Module 2: Basics of Feedforward Neural Networks

Dr. Somdyuti Paul

Feedforward Neural Network

- A general feedforward neural network (NN) architecture consists of an input layer, one or more hidden layers and an output layer, connected sequentially.
- The connections between neurons in a feedforward neural network does not form cycles or loops
- The number of layers in a feedforward NN determines its *depth*.
- The number of units in a given layer of the NN determines its *width*.

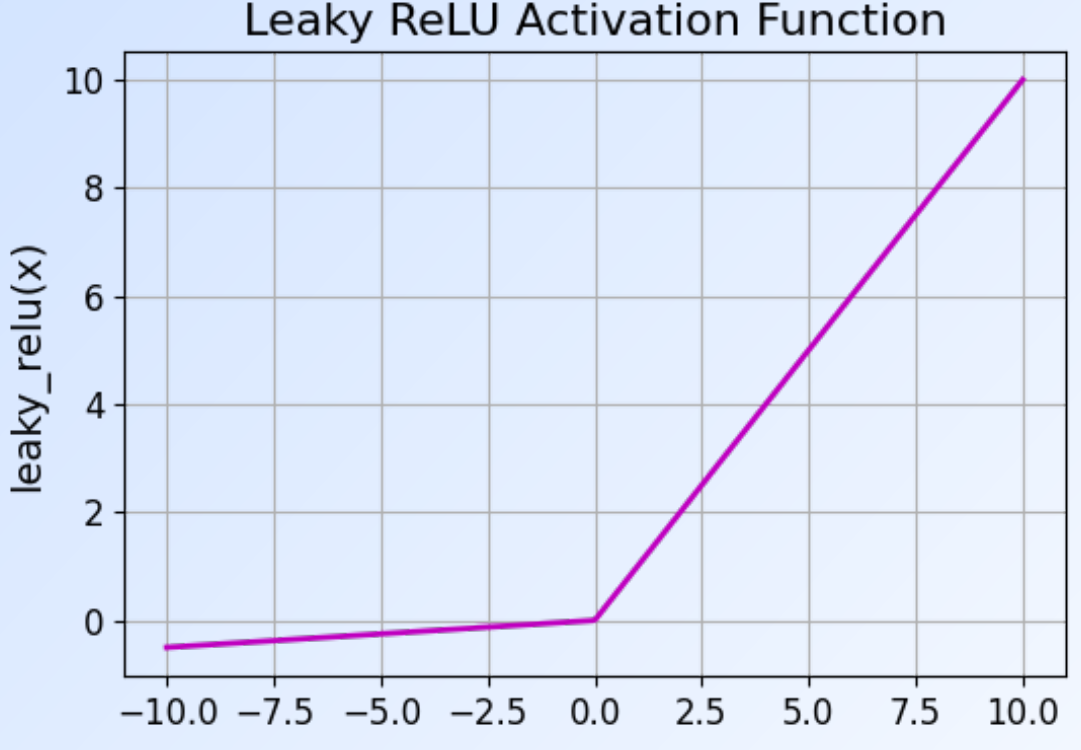
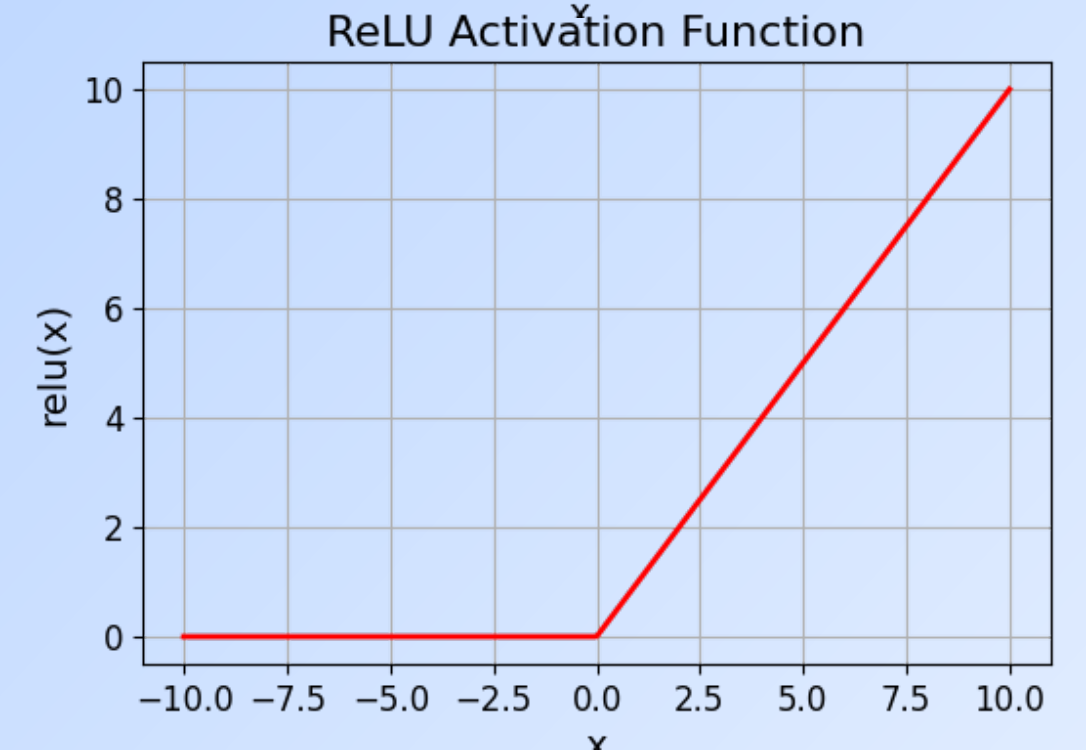
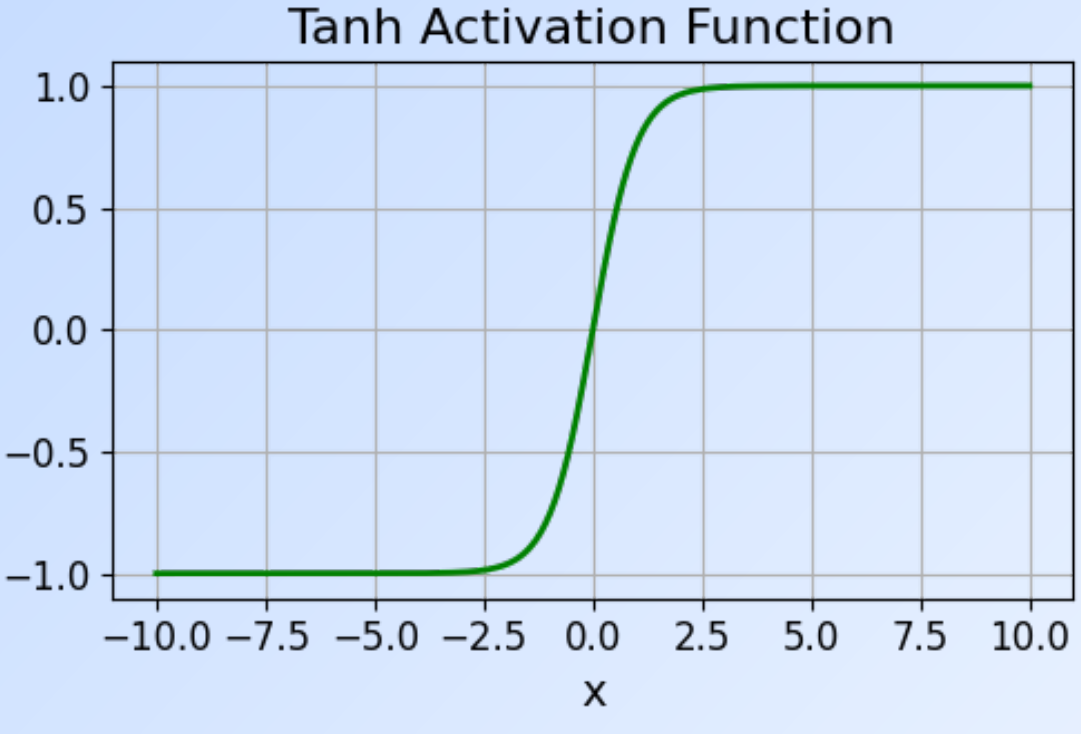
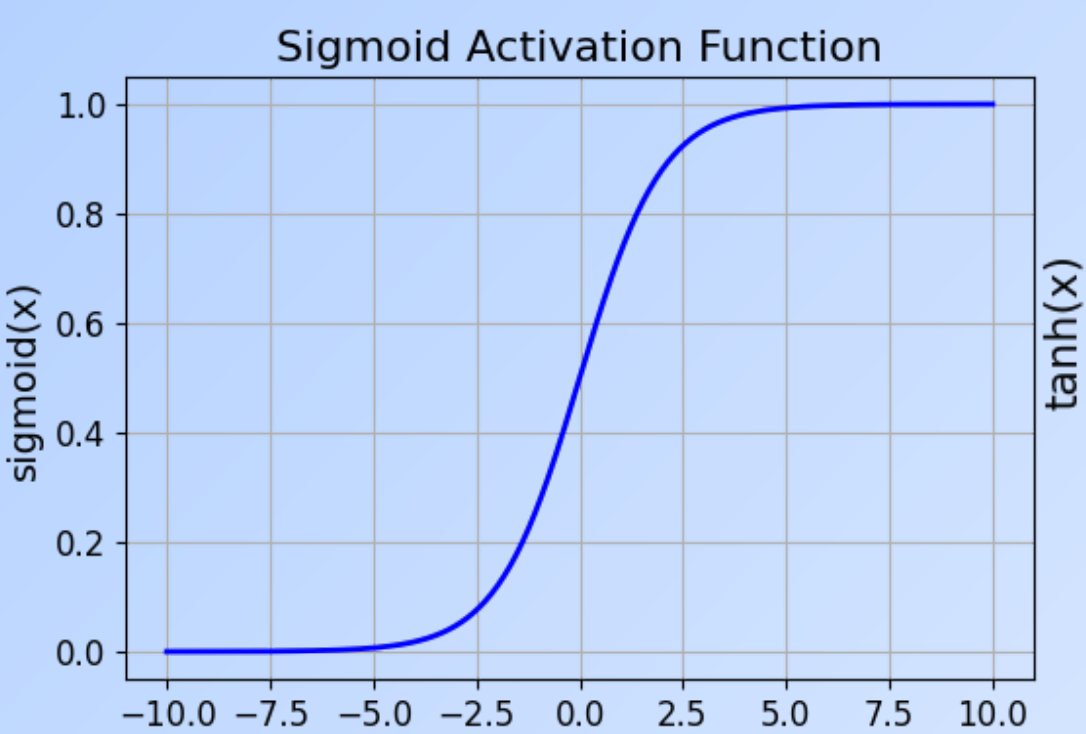


Weights, Biases and Activation Functions

- Each neuron in a neural network is associated with the following:
 - Weights: parameters that control the strength of the connections of a neuron to its inputs.
 - Bias: parameters that shift the decision boundary of each neuron, imparting flexibility to the model.
 - Activation function: activation functions are applied to the weighted linear combination of the inputs to introduce non-linearity to the network to enable it to learn complex data relationships and intricate patterns.
- The total number of trainable parameters is total number of weights and biases of all neurons in the network.
- The weights and bias of each neuron of a neural network are adjusted during training to minimize the error between predicted and actual outputs.

Common Activation Functions

Name	Functional form	Common usage
Linear	$g(z) = z$	Output layer
ReLU	$g(z) = \max\{0, z\}$	Hidden layers
Logistic	$g(z) = \frac{1}{1 + e^{-z}}$	Output layer Hidden layer in legacy architectures
Hyperbolic Tangent	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Hidden layer in legacy architectures
Softmax	$g(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$	Output layers



Neural Networks as Universal Function Approximators

A feedforward network with a single hidden layer and any *squashing* activation function can approximate any continuous function defined on compact domain with any precision, provided that there are enough hidden units.

Universal approximation theorem (G. Cybenko, 1989):

Let I_n be the unit cube $[0,1]^n$ and $C(I_n)$ be the space of all continuous functions on I_n , and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a sigmoidal function that satisfies:

$$\sigma(z) \rightarrow \begin{cases} 1 & \text{if } z \rightarrow \infty \\ 0 & \text{if } z \rightarrow -\infty \end{cases}$$

Then for every $f(\mathbf{x}) \in C(I_n)$ and $\epsilon > 0$, $\exists g(\mathbf{x}) = \sum_i \alpha_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i)$, with σ being a sigmoidal function such that

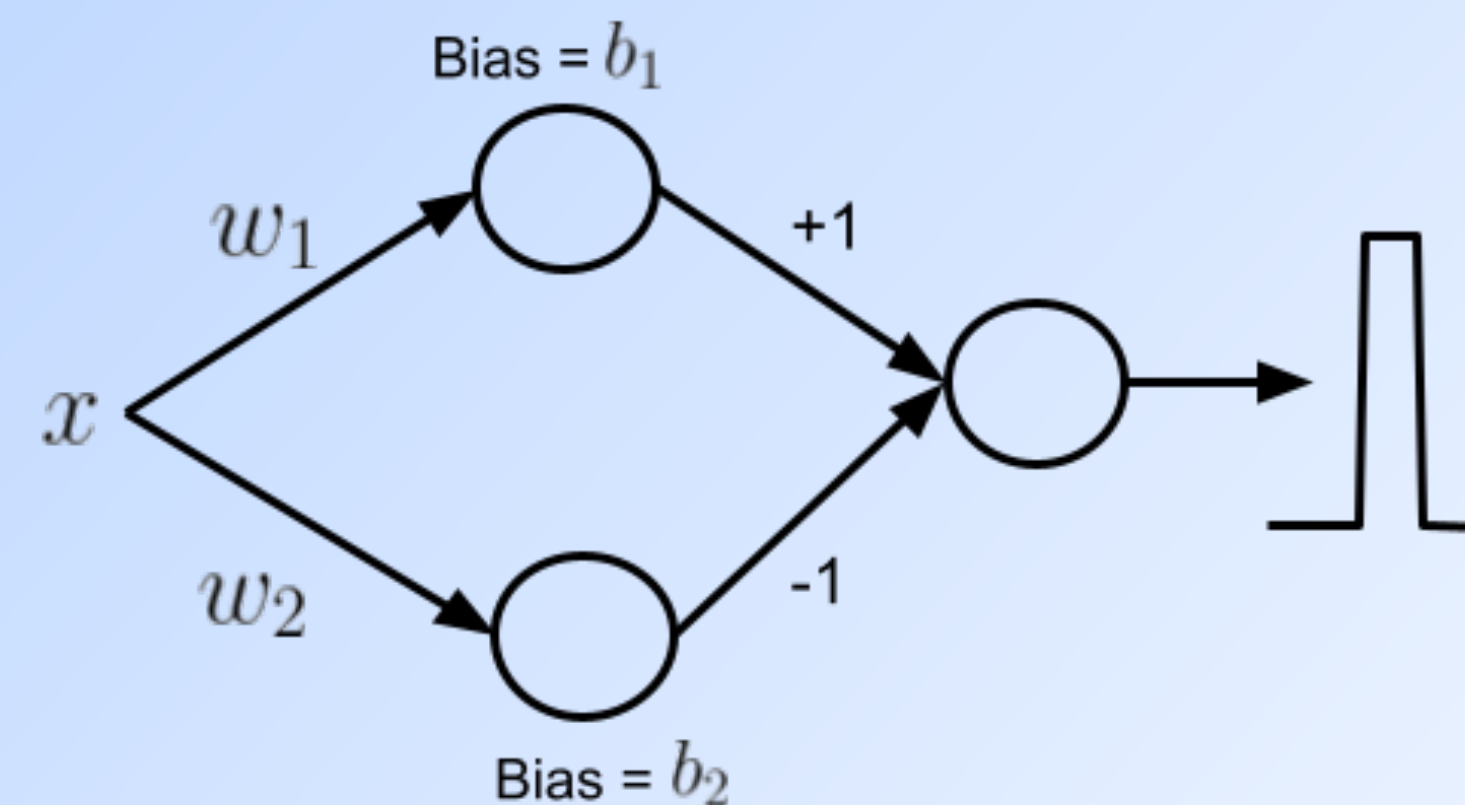
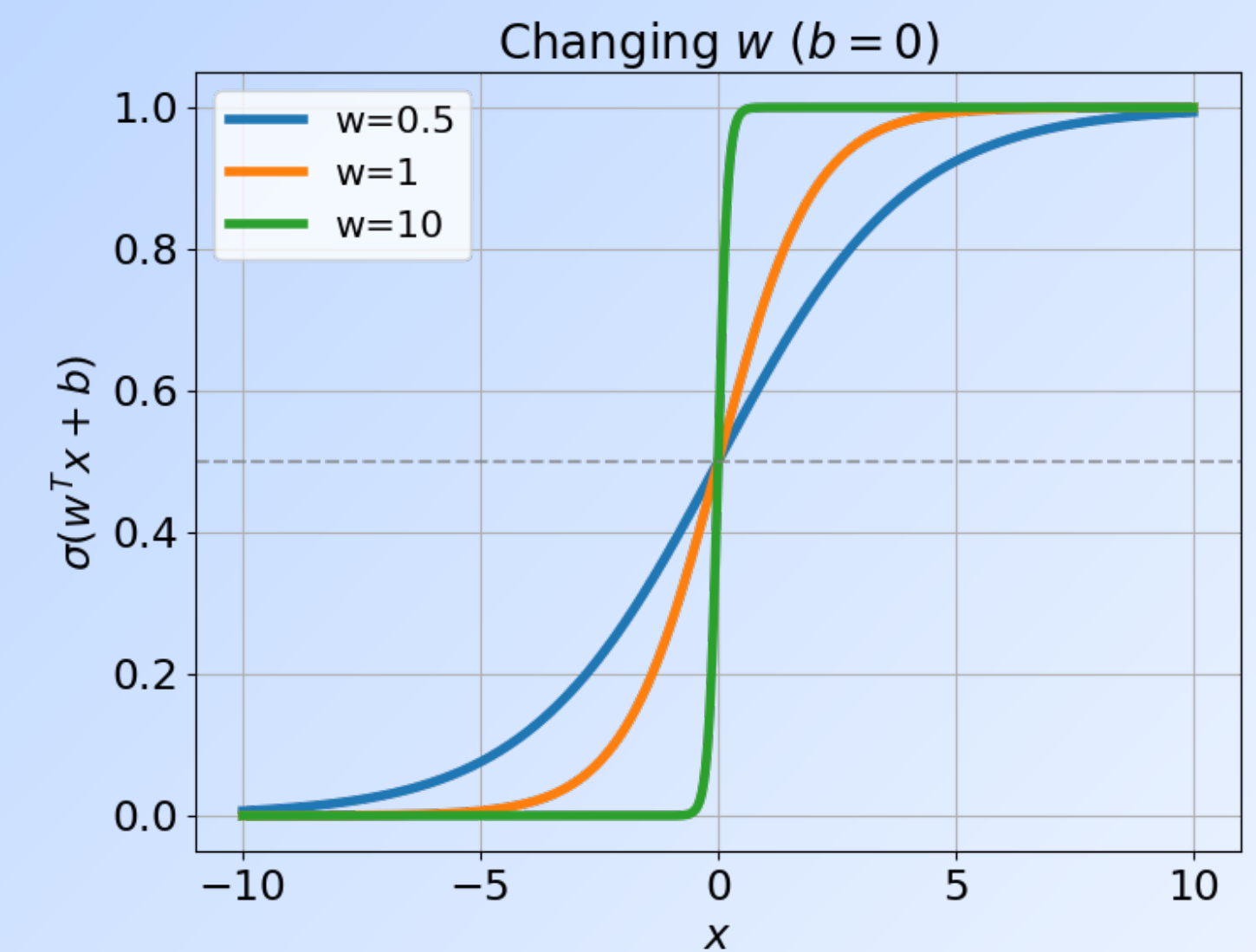
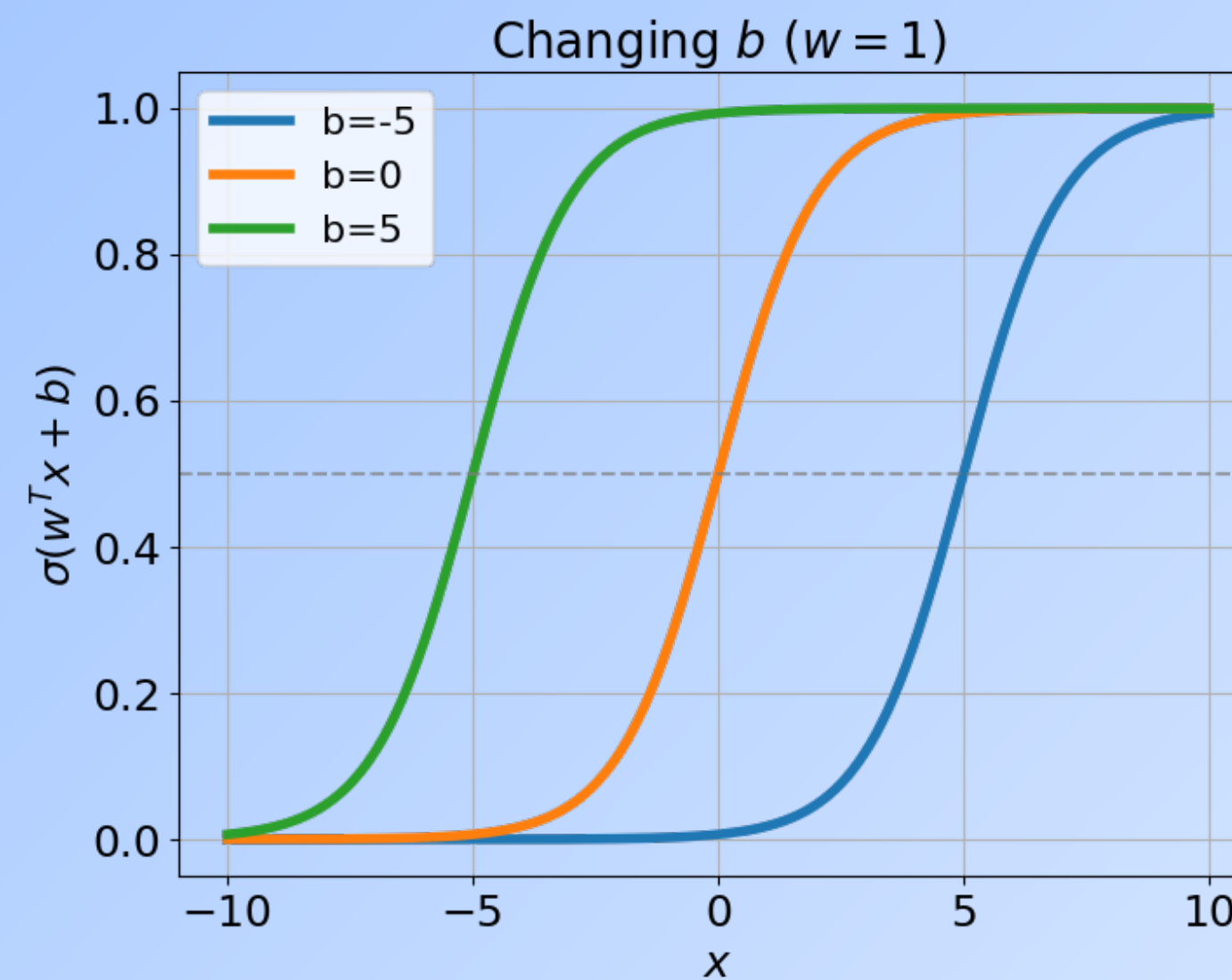
$$\sup \{ |g(\mathbf{x}) - f(\mathbf{x})| : \mathbf{x} \in I_n \} < \epsilon$$

The universal approximation property of neural networks holds for any arbitrary non-linear activation function (K. Hornik, 1991)

Neural Networks as Universal Function Approximators

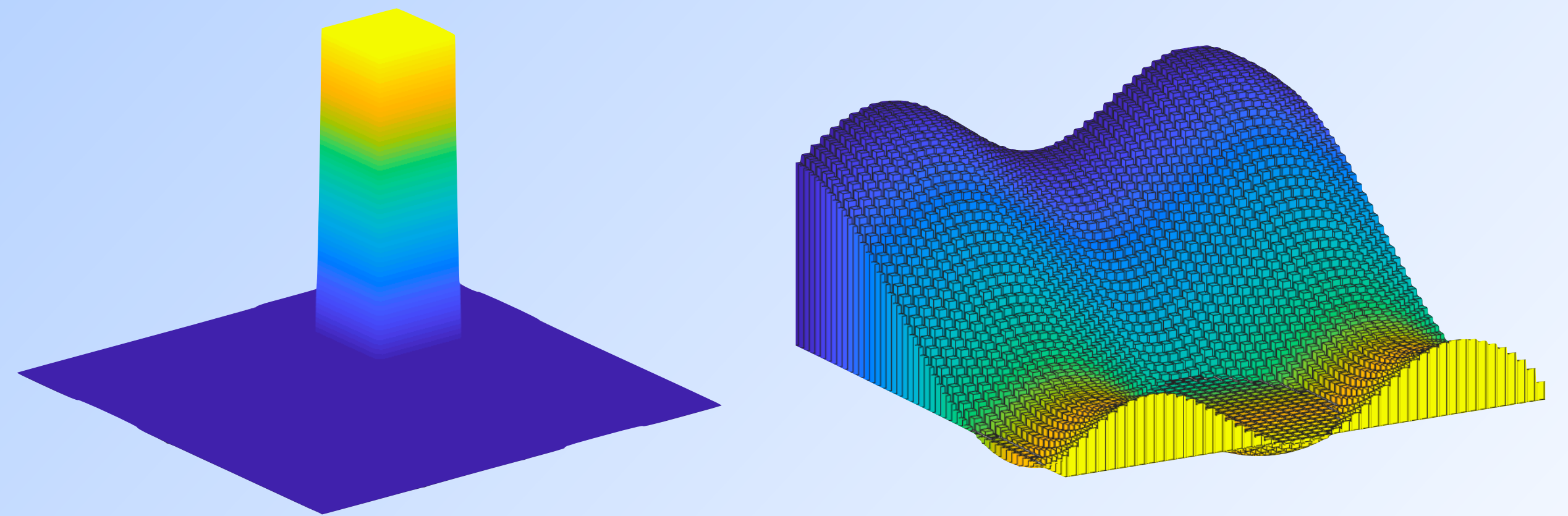
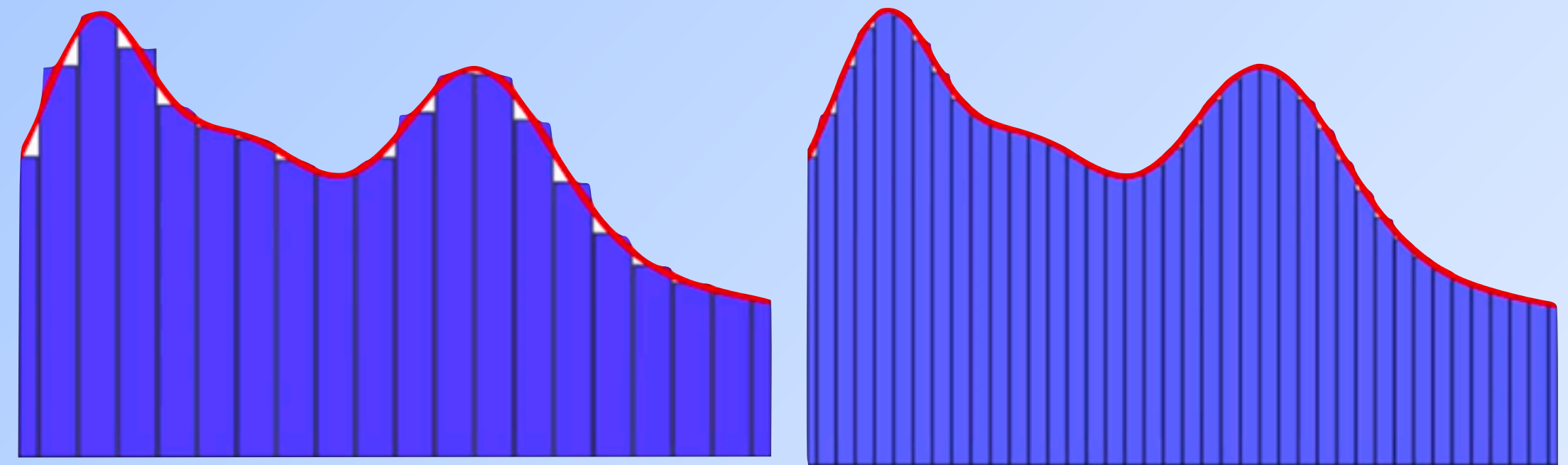
Intuition

- Sigmoid functions could be used to approximate a step function.
- A network with 3 neurons can compute a pulse.



Neural Networks as Universal Function Approximators

- A train of pulses can approximate a continuous function to any arbitrary precision.
- The concept generalizes to higher dimensions.



Parametric Supervised Learning Framework for Neural Networks

To train a neural network in a parametric, supervised learning framework, the following must be defined:

- Dataset: $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^M$
- Model: $\hat{y}^{(i)} = h_{\theta}(\mathbf{x}^{(i)})$.
- Cost function: $J(\theta) = \sum_{i=1}^M \ell(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$
- Optimizer: $\theta_{new} = \theta - \alpha \nabla_{\theta} J(\theta)$
- Performance metrics: MSE, accuracy, precision, recall, etc.

Computational Graph of a Neural Network

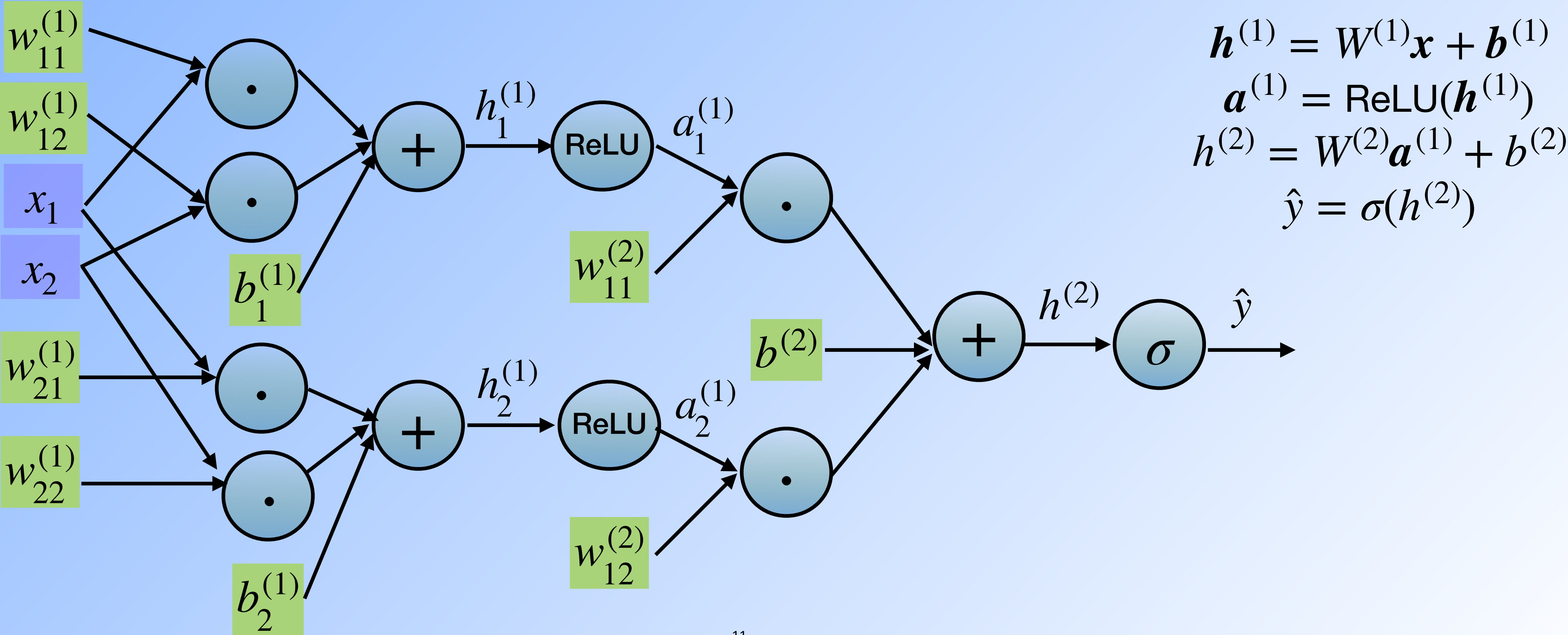
- A computational graph is a graphical representation of the flow of data and operations.
- A neural network can be represented by a directed graph such that
 - Nodes represent operations (or variables)
 - Edges represent data flow (or operations)
 - The graph is defined with respect to a set of allowable operations.

Computational Graph of a Neural Network

- The computational graph facilitates two key processes in neural networks
 - **Forward pass**: involves performing sequential computations as defined by the computational graph to determine the output of the network as well as the associated cost function.
 - **Backpropagation**: involves computing the gradients of the cost with respect to the weights of each layer by traversing the computational graph backward node by node, starting with the output node and ending at the input node.
- The computational graph also enables the following:
 - Optimization for efficient execution: shared subgraphs could be computed once and re-used.
 - Parallelization: independent operations in the graph could be parallelized over multiple GPUs.

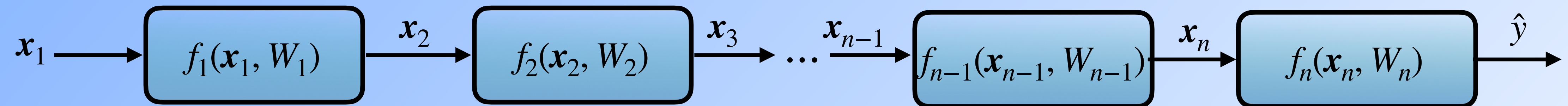
Computational Graph of a Neural Network

Example: Computational graph of a 2 layer neural network:



Learning Neural Network Parameters Using Backpropagation

A neural network may be represented as a repeated composition of functions



$$\hat{y} = f_n(f_{n-1}(\cdots(f_2(f_1(\mathbf{x}_1, W_1), W_2)\cdots), W_{n-1}), W_n)$$

$$J = \sum_i^M \ell(y^{(i)}, \hat{y}^{(i)}) = \sum_{i=1}^M \ell(y^{(i)}, f_n(f_{n-1}(\cdots(f_2(f_1(\mathbf{x}_1^{(i)}, W_1), W_2)\cdots), W_{n-1}), W_n))$$

Backpropagation involves the recursive application of the chain rule of derivatives:

$$\frac{\partial J}{\partial W_j} = \frac{\partial J}{\partial \hat{y}} \frac{\partial f_n(\mathbf{x}_n, W_n)}{\partial \mathbf{x}_n} \frac{\partial f_{n-1}(\mathbf{x}_{n-1}, W_{n-1})}{\partial \mathbf{x}_{n-1}} \cdots \frac{\partial f_{j+1}(\mathbf{x}_{j+1}, W_{j+1})}{\partial \mathbf{x}_{j+1}} \frac{\partial f_j(\mathbf{x}_j, W_j)}{\partial W_j} = \frac{\partial J}{\partial \mathbf{x}_{j+1}} \frac{\partial f_j(\mathbf{x}_j, W_j)}{\partial W_j}$$

Gradient Flow

$$\frac{\partial J}{\partial W_j} = \frac{\partial J}{\partial \mathbf{x}_{j+1}} \frac{\partial f_j(\mathbf{x}_j, W_j)}{\partial W_j} = \frac{\partial J}{\partial \hat{y}} \frac{\partial f_n(\mathbf{x}_n, W_n)}{\partial \mathbf{x}_n} \frac{\partial f_{n-1}(\mathbf{x}_{n-1}, W_{n-1})}{\partial \mathbf{x}_{n-1}} \dots \frac{\partial f_{j+1}(\mathbf{x}_{j+1}, W_{j+1})}{\partial \mathbf{x}_{j+1}} \frac{\partial f_j(\mathbf{x}_j, W_j)}{\partial W_j}$$

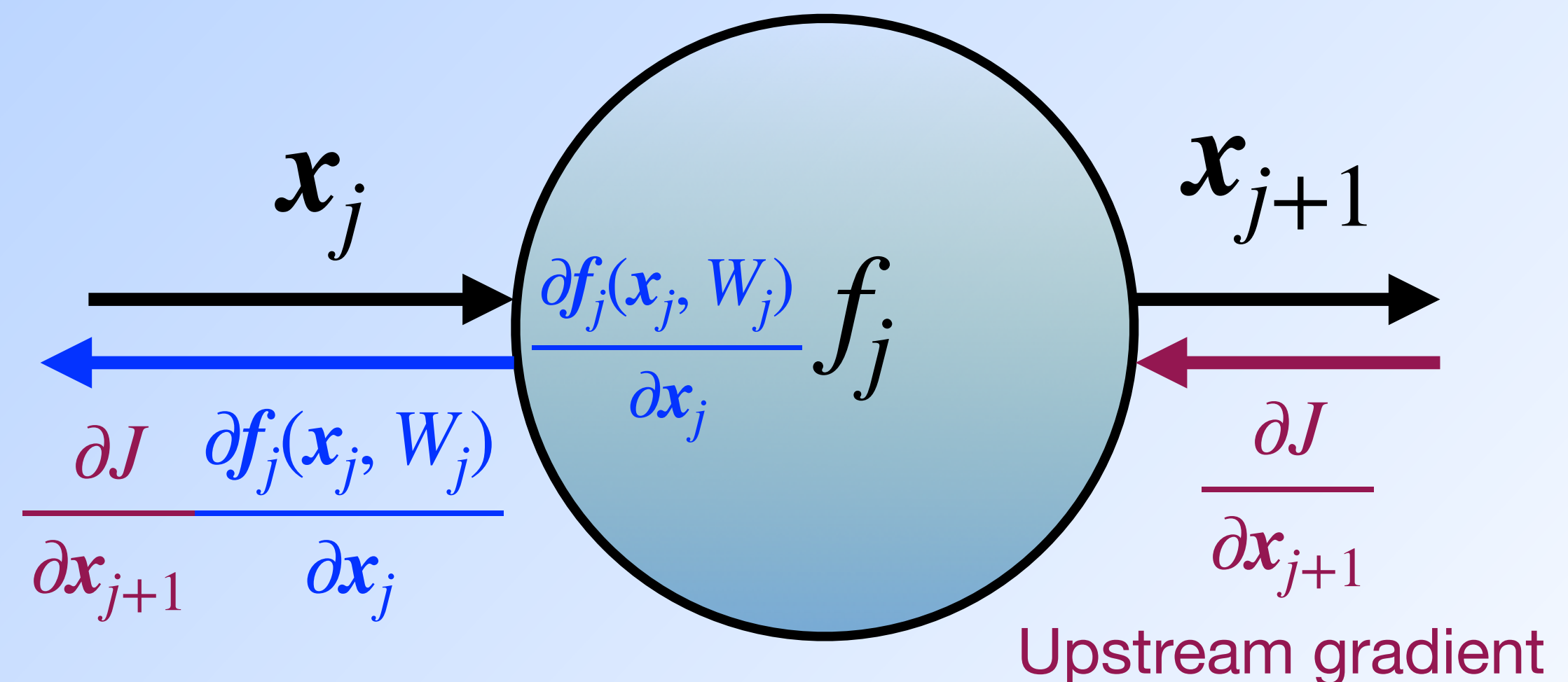
Computed recursively

If the gradient $\frac{\partial J}{\partial \mathbf{x}_{j+1}}$ is available, the gradient of the preceding

layer can be computed as follows:

$$\frac{\partial J}{\partial \mathbf{x}_j} = \frac{\partial J}{\partial \mathbf{x}_{j+1}} \frac{\partial \mathbf{x}_{j+1}}{\partial \mathbf{x}_j} = \frac{\partial J}{\partial \mathbf{x}_{j+1}} \frac{\partial f_j(\mathbf{x}_j, W_j)}{\partial \mathbf{x}_j}$$

Downstream gradient = Upstream gradient × Local gradient



Downstream gradient = Upstream gradient × Local gradient

Gradient Flow

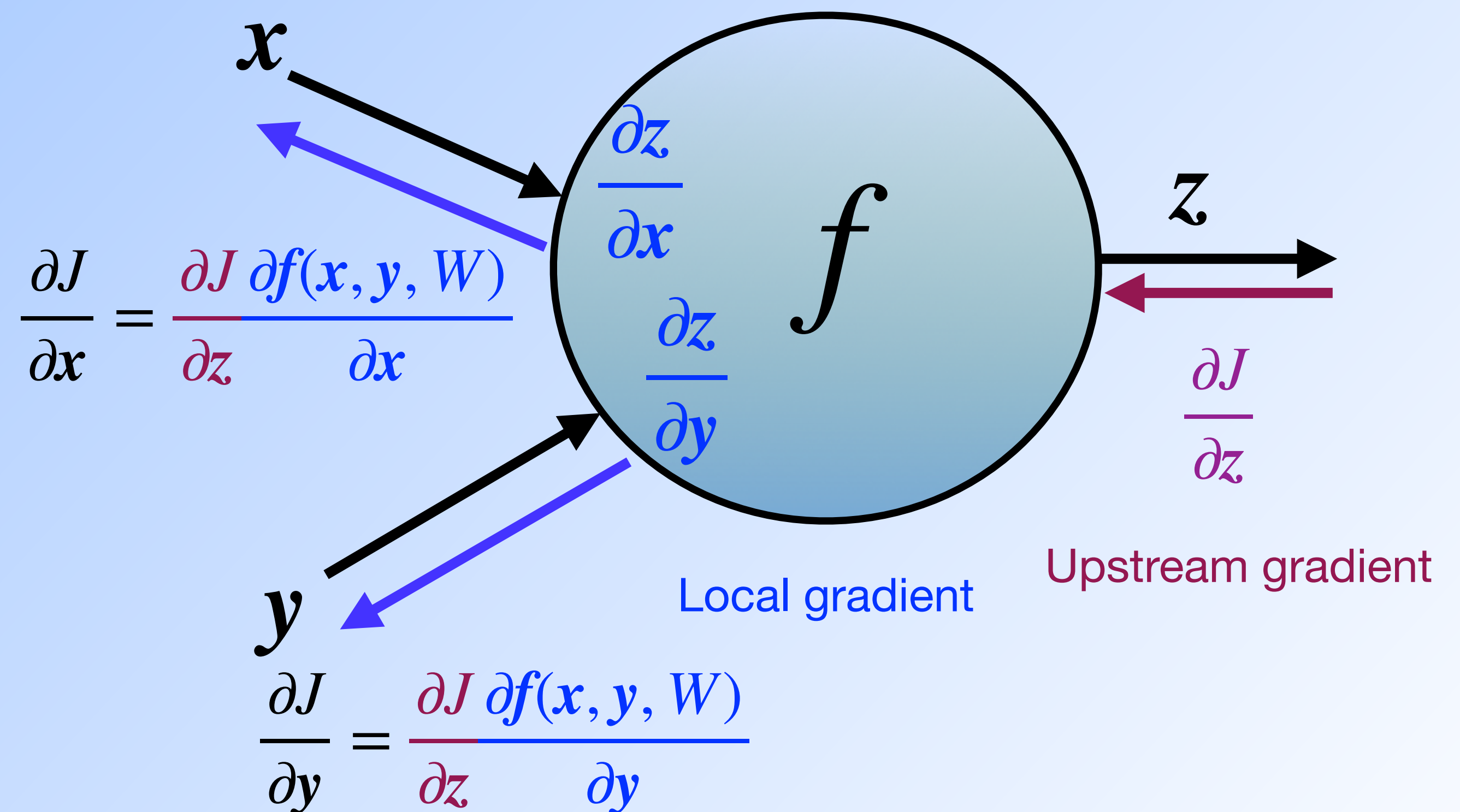
When a layer has more than one input, a local gradient is computed with respect to each input.

$$z = f(x, y, W)$$

$$\frac{\partial J}{\partial x} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial x} = \frac{\partial J}{\partial z} \frac{\partial f(x, y, W)}{\partial x}$$

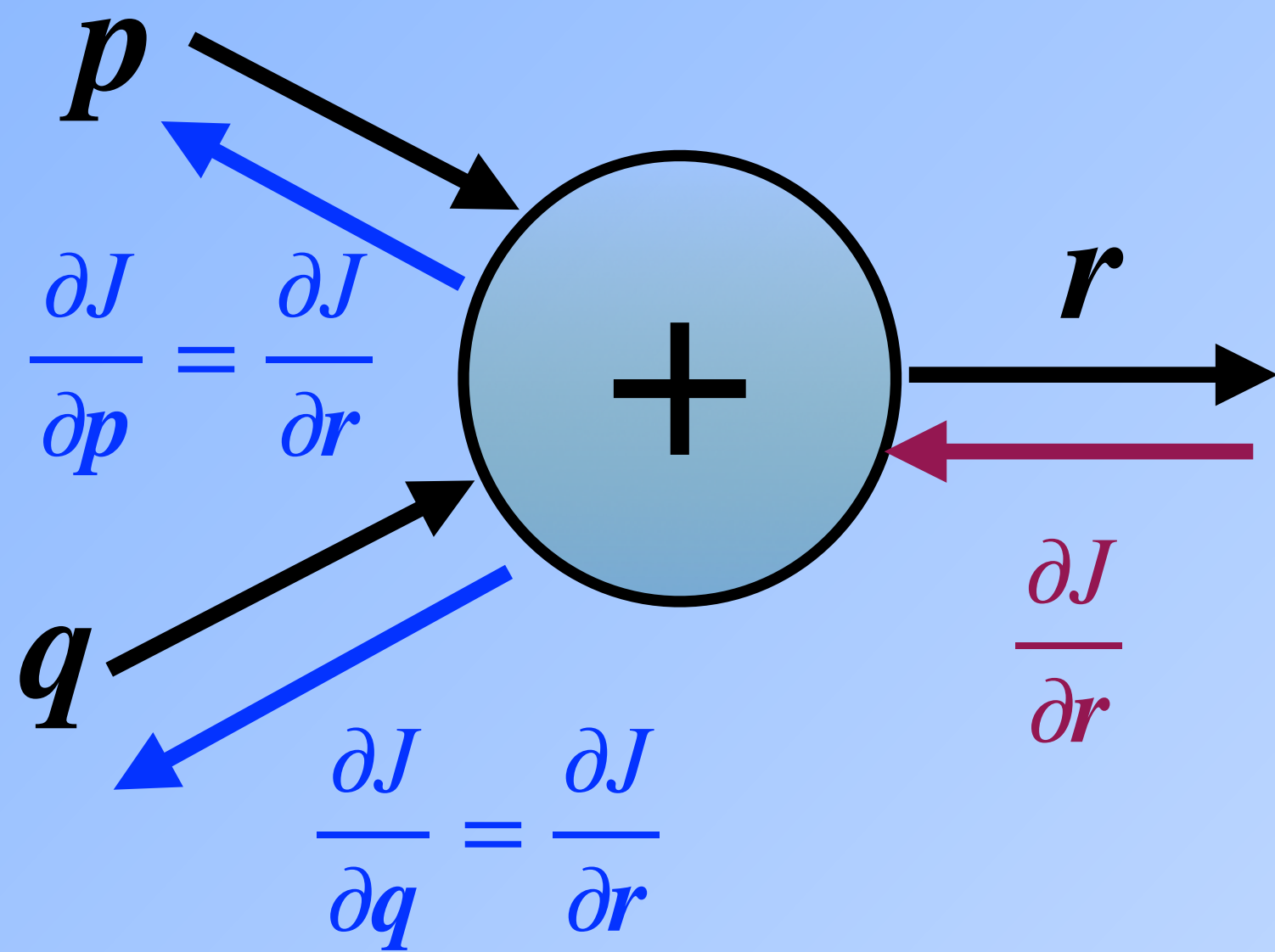
$$\frac{\partial J}{\partial y} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial y} = \frac{\partial J}{\partial z} \frac{\partial f(x, y, W)}{\partial y}$$

**Downstream gradient = Upstream gradient ×
Local gradient**

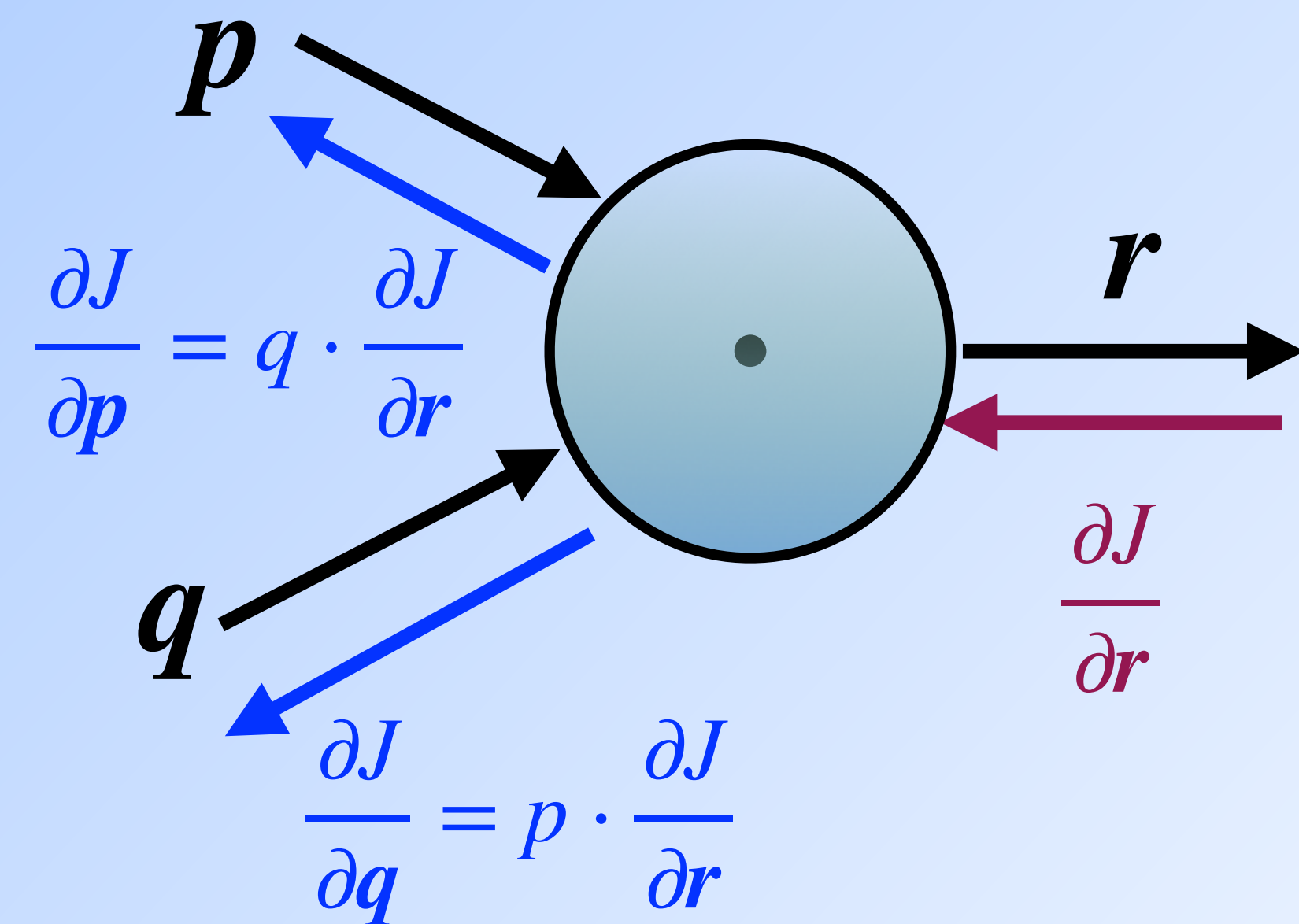


Downstream gradients

Gradient Flow Patterns

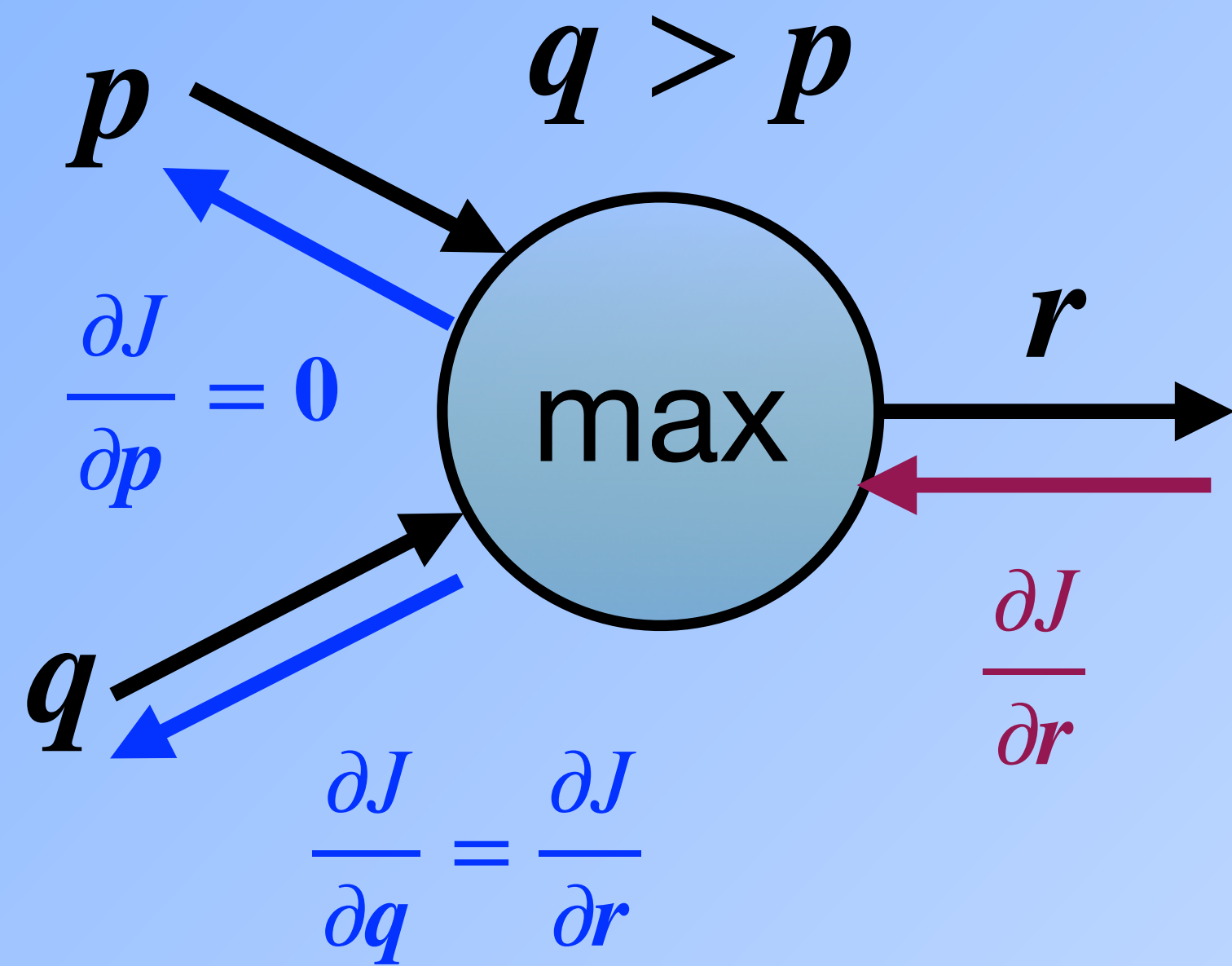


Addition: gradient distributor

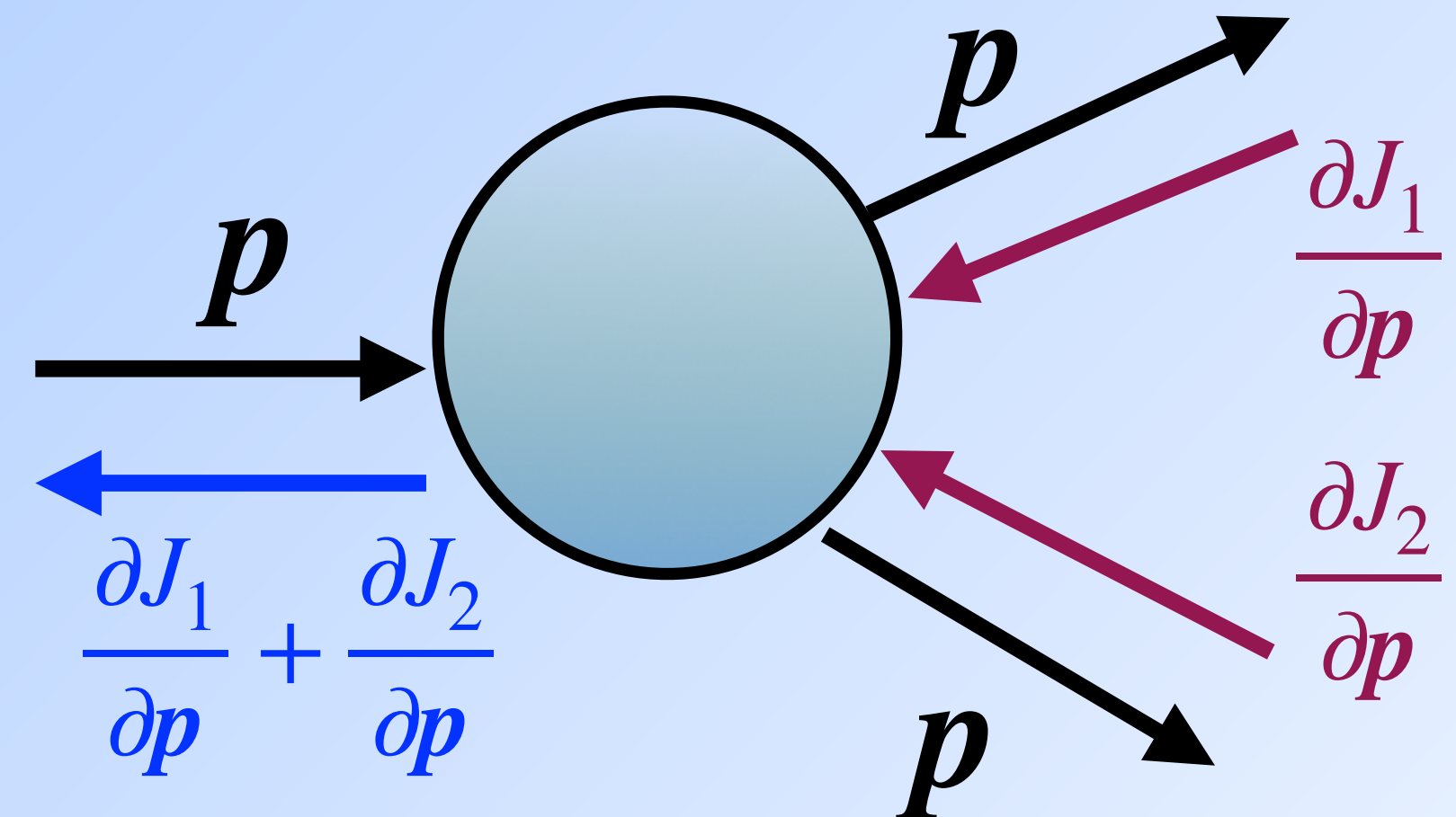


Multiplication: gradient switcher

Gradient Flow Patterns



Max: gradient router



Copy: gradient adder

Vector Derivatives

Partial derivative: $x, y \in \mathbb{R}, f(x, y) \in \mathbb{R}, \frac{\partial f}{\partial x} \in \mathbb{R}$

Gradient: $\mathbf{x} \in \mathbb{R}^n, f(\mathbf{x}) \in \mathbb{R}, \nabla_{\mathbf{x}} f \in \mathbb{R}^n$

$$\nabla_{\mathbf{x}} f = \left[\frac{\partial f}{\partial x_1} \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

Note: the gradient $\nabla_{\mathbf{x}} f$ can be written as a row vector (using numerator layout) or as a column vector (using denominator layout). Here, we are using the former convention.

Jacobian: $\mathbf{x} \in \mathbb{R}^n, \mathbf{f}(\mathbf{x}) \in \mathbb{R}^m, \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n}$ (numerator layout)

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Gradients for Linear Functions

Each layer of a feedforward neural network performs linear operations of the form:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad (\mathbf{x} \in \mathbb{R}^n, \mathbf{W} \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m)$$

The gradients with respect to the cost function on backpropagation through this linear function are then computed as:

$$\frac{\partial J}{\partial \mathbf{x}} = \frac{\partial J}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{x}}$$

$$\frac{\partial J}{\partial \mathbf{W}} = \frac{\partial J}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$$

$$\frac{\partial J}{\partial \mathbf{b}} = \frac{\partial J}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

$$\frac{\partial z_i}{\partial x_j} = W_{ij}$$

$$\text{Let } \frac{\partial J}{\partial \mathbf{z}} = \boldsymbol{\delta} \text{ (gradient of shape } 1 \times m)$$

$$\text{Let } \frac{\partial \mathbf{z}}{\partial \mathbf{b}} = \mathbf{I} \text{ (} m \times m \text{ identity matrix)}$$

$$\therefore \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{W} \text{ (Jacobian of shape } m \times n)$$

$$\frac{\partial z_i}{\partial W_{ij}} = x_j$$

$$\frac{\partial J}{\partial \mathbf{b}} = \frac{\partial J}{\partial \mathbf{z}} \mathbf{I}$$

$$\frac{\partial J}{\partial \mathbf{x}} = \frac{\partial J}{\partial \mathbf{z}} \mathbf{W}$$

$$\frac{\partial J}{\partial W_{ij}} = \frac{\partial J}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial W_{ij}} = \delta_i x_j$$

$$\frac{\partial J}{\partial \mathbf{W}} = \boldsymbol{\delta}^T \mathbf{x}^T$$

Gradients for Pointwise Activation Functions

Each hidden layer of feedforward neural network applies element-wise activation functions on the output of the linear operations as follows:

$$\mathbf{a} = \sigma(\mathbf{z}) \quad (\mathbf{z} \in \mathbb{R}^m)$$

$$\frac{\partial J}{\partial \mathbf{z}} = \frac{\partial J}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \frac{\partial J}{\partial \mathbf{a}} \odot [\sigma'(z_1), \sigma'(z_2), \dots, \sigma'(z_m)]$$

Examples:

tanh activation

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\sigma'(z) = 1 - \sigma(z)^2$$

Logistic activation

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

ReLU activation

$$\sigma(z) = \max(0, z)$$

$$\sigma'(z) = \mathbf{1}_{z>0}$$

Backpropagation Through a Neural Network

- Consider the following 2 layer feedforward neural network:

$$\begin{aligned} \mathbf{h}^{(1)} &= W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} & (\mathbf{x} \in \mathbb{R}^{n \times 1}, W^{(1)} \in \mathbb{R}^{m \times n}, \mathbf{b}^{(1)} \in \mathbb{R}^{m \times 1}) \\ \mathbf{a}^{(1)} &= \text{ReLU}(\mathbf{h}^{(1)}) & (\mathbf{h}^{(1)} \in \mathbb{R}^{m \times 1}) \end{aligned}$$

$$\begin{aligned} \mathbf{h}^{(2)} &= W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} & (\mathbf{a} \in \mathbb{R}^{m \times 1}, W^{(2)} \in \mathbb{R}^{k \times m}, \mathbf{b}^{(2)} \in \mathbb{R}^{k \times 1}) \\ \hat{\mathbf{y}} &= \sigma(\mathbf{h}^{(2)}) & (\mathbf{h}^{(2)} \in \mathbb{R}^{k \times 1}) \end{aligned}$$

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^k y_j \log(\hat{y}_j) + (1 - y_j) \log(1 - \hat{y}_j)$$

- The back propagation through this neural network works as follows:

Step 1: Gradient of the loss w.r.t the output

$$\frac{\partial \ell}{\partial \hat{y}_j} = \frac{\hat{y}_j - y_j}{\hat{y}_j(1 - \hat{y}_j)} \quad \left(\frac{\partial \ell}{\partial \hat{y}_j} \in \mathbb{R}^{(1 \times 1)} \right)$$

Backpropagation Through a Neural Network

Step 2: Gradients of the loss w.r.t output layer parameters:

$$\frac{\partial \ell}{\partial h_j^{(2)}} = \frac{\partial \ell}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial h_j^{(2)}} = \frac{\partial \ell}{\partial \hat{y}_j} \sigma(h_j^{(2)}) (1 - \sigma(h_j^{(2)})) = \frac{\hat{y}_j - y_j}{\hat{y}_j(1 - \hat{y}_j)} \hat{y}_j(1 - \hat{y}_j) = \hat{y}_j - y_j$$

$$\frac{\partial \ell}{\partial \mathbf{h}^{(2)}} = (\hat{\mathbf{y}} - \mathbf{y})^T \quad \left(\frac{\partial \ell}{\partial \mathbf{h}^{(2)}} \in \mathbb{R}^{(1 \times k)} \right)$$

$$\frac{\partial \ell}{\partial W^{(2)}} = \left(\frac{\partial \ell}{\partial \mathbf{h}^{(2)}} \right)^T \frac{\partial \mathbf{h}^{(2)}}{\partial W^{(2)}} = (\hat{\mathbf{y}} - \mathbf{y})(\mathbf{a}^{(1)})^T \quad \left(\frac{\partial \ell}{\partial W^{(2)}} \in \mathbb{R}^{(k \times m)} \right)$$

$$\frac{\partial \ell}{\partial \mathbf{b}^{(2)}} = \frac{\partial \ell}{\partial \mathbf{h}^{(2)}} \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{b}^{(2)}} = (\hat{\mathbf{y}} - \mathbf{y})^T I_{k \times k} \quad \left(\frac{\partial \ell}{\partial \mathbf{b}^{(2)}} \in \mathbb{R}^{(1 \times k)} \right)$$

Backpropagation Through a Neural Network

Step 3: Gradients of the loss w.r.t hidden layer parameters:

$$\frac{\partial \ell}{\partial \mathbf{a}^{(1)}} = \frac{\partial \ell}{\partial \mathbf{h}^{(2)}} \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{a}^{(1)}} = \frac{\partial \ell}{\partial \mathbf{h}^{(2)}} \mathbf{W}^{(2)} = (\hat{\mathbf{y}} - \mathbf{y})^T \mathbf{W}^{(2)} \quad \left(\frac{\partial \ell}{\partial \mathbf{a}^{(1)}} \in \mathbb{R}^{(1 \times m)} \right)$$

$$\frac{\partial \ell}{\partial \mathbf{h}^{(1)}} = \frac{\partial \ell}{\partial \mathbf{a}^{(1)}} \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{h}^{(1)}} = (\hat{\mathbf{y}} - \mathbf{y})^T \mathbf{W}^{(2)} \odot \text{ReLU}'(\mathbf{h}^{(1)}) \quad \left(\frac{\partial \ell}{\partial \mathbf{h}^{(1)}} \in \mathbb{R}^{(1 \times m)} \right)$$

$$\frac{\partial \ell}{\partial \mathbf{W}^{(1)}} = \left(\frac{\partial \ell}{\partial \mathbf{h}^{(1)}} \right)^T \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{W}^{(1)}} = \left((\hat{\mathbf{y}} - \mathbf{y})^T \mathbf{W}^{(2)} \odot \text{ReLU}'(\mathbf{h}^{(1)}) \right)^T \mathbf{x}^T \quad \left(\frac{\partial \ell}{\partial \mathbf{W}^{(1)}} \in \mathbb{R}^{(m \times n)} \right)$$

$$\frac{\partial \ell}{\partial \mathbf{b}^{(1)}} = \frac{\partial \ell}{\partial \mathbf{h}^{(1)}} \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{b}^{(1)}} = (\hat{\mathbf{y}} - \mathbf{y})^T \mathbf{W}^{(2)} \odot \text{ReLU}'(\mathbf{h}^{(1)}) \mathbf{I}_{m \times m} \quad \left(\frac{\partial \ell}{\partial \mathbf{b}^{(1)}} \in \mathbb{R}^{(1 \times m)} \right)$$

Backpropagation Using Computational Graph

